

AI Data Intelligence Pipeline Architecture

Overview

This project implements a scalable Data Intelligence Pipeline for collecting, normalizing, enriching, and exporting data from multiple AI ecosystem sources.

The system currently ingests:

- * Research Papers (Arxiv)
- * AI Products (Hugging Face Models)
- * AI Jobs (WeWorkRemotely RSS)
- * AI News (TechCrunch AI Feed)

The pipeline exports structured data into CSV and JSON formats for downstream analytics and reporting.

System Architecture

Data Sources

- * Arxiv API
- * Hugging Face Models API
- * WeWorkRemotely RSS
- * TechCrunch AI RSS

Data Flow

Source Collection



Data Cleaning



Entity Resolution



Schema Mapping



Export Layer



CSV / JSON Outputs

Research Paper Pipeline

Source:

Arxiv API

Collected Fields:

- * Title
- * Authors
- * Publication Date
- * Paper URL

Additional Enrichment:

- * GitHub Repository Lookup
- * GitHub Stars Extraction

Output:

research_papers.csv

research_papers.json

Product Pipeline

Source:

Hugging Face Models API

Collected Fields:

- * Product Name
- * Downloads

* Likes

Output:

products.csv

Scalability:

Pagination support enables collection of thousands of product records.

Job Monitoring Pipeline

Source:

WeWorkRemotely RSS

Collected Fields:

* Company

* Job Title

* URL

* Published Date

* Remote Flag

* Role Family

Freshness Tracking:

Jobs are filtered using publication timestamps to ensure only recent records are processed.

Output:

jobs.csv

News Monitoring Pipeline

Source:

TechCrunch AI Feed

Collected Fields:

* Title

* URL

* Published Date

Freshness Strategy:

Publication timestamps are parsed and compared against current UTC time.

Output:

news.csv

Entity Resolution

Purpose:

Normalize multiple representations of the same organization.

Examples:

Open AI → OpenAI

OpenAI Inc. → OpenAI

Anthropic AI → Anthropic

DeepMind → Google DeepMind

Output:

entity_mapping.csv

Handling 429 Rate Limits

Strategy:

- * Retry mechanism

- * Exponential backoff

* Random jitter

Example:

Attempt 1

↓

429

↓

Wait 1 second

↓

Retry

↓

Wait 2 seconds

↓

Retry

This prevents service overload and improves reliability.

Handling 413 Payload Too Large

Strategy:

- * Intelligent chunking
- * Content truncation
- * Context size limits

Large documents are split into smaller payloads before being sent to LLM services.

This prevents request failures caused by model context limits.

LLM Fallback Chain

Primary:

Gemini Flash

Fallback 1:

Groq Llama

Fallback 2:

DeepSeek

Workflow:

Gemini



Failure



Groq



Failure



DeepSeek



Success

This ensures high availability.

Scale Strategy (500,000+ Records)

To scale beyond hundreds of thousands of records:

- * Async Crawlers (asyncio)
- * Distributed Workers
- * Queue-Based Processing
- * Batch Exports
- * Parallel API Requests

No application code changes are required.

Only infrastructure scaling is needed.

Freshness Tracking

Each record stores:

- * Publication Timestamp
- * Collection Timestamp

Duplicate processing can be avoided using:

- * URL hashing
- * Timestamp comparison
- * Source-specific identifiers

Storage Strategy

Structured Data:

PostgreSQL

Relationship Graph:

Neo4j

Vector Search:

Qdrant

Benefits:

- * Fast querying
- * Relationship discovery
- * Semantic search capabilities

Deliverables Generated

Current Outputs:

- * research_papers.csv
- * research_papers.json

- * products.csv
- * jobs.csv
- * news.csv
- * entity_mapping.csv

Future Enhancements

- * Startup Ingestion at Scale
- * Product-to-Company Linking
- * Advanced Deduplication
- * Distributed Crawling Infrastructure
- * Automated Google Sheets Export

Conclusion

The system demonstrates a production-oriented Data Intelligence Pipeline with multi-source ingestion, structured schema generation, entity resolution, export functionality, retry handling, LLM fallback orchestration, and scalability planning for large-scale intelligence collection.